

Enunciado del Proyecto: Calabozo Místico

1. Descripción

Desarrolle un videojuego 2D basado en un tablero tipo grid (cuadrícula) donde un personaje debe escapar de un calabozo. Para lograrlo, el jugador deberá recolectar 4 llaves escondidas en el mapa y luego abrir la puerta de salida. Sin embargo, el calabozo está vigilado por 3 dragones, cada uno con un comportamiento de persecución distinto, cuya lógica debe emular “estilos” de caza diferentes (descritos abajo).

Además del juego en sí, el sistema debe permitir guardar/cargar el estado de la partida, y al completar el juego debe ofrecer una opción para ver la repetición (replay) de la partida. El proyecto se realizará con el lenguaje de programación Python.

2. Características

2.1 Objetivo

- El jugador controla un personaje en un tablero tipo grid.
- El objetivo es:
 1. Recolectar las 4 llaves.
 2. Llegar a la puerta de salida.
 3. Abrir la puerta (solo si ya tiene las 4 llaves) y escapar.

2.2 Reglas y mecánicas

- El tablero está compuesto por celdas (por ejemplo: paredes, piso, llaves, puerta, dragones, personaje).
- El personaje se mueve celda a celda (4 direcciones: arriba/abajo/izquierda/derecha), sin atravesar paredes u obstáculos.
- Si un dragón alcanza al personaje (misma celda), la partida termina (o se reinicia, según su diseño, pero debe estar claramente definido).

2.3 Dragones: comportamiento algorítmico

Debe implementar **tres dragones** con estrategias diferentes. La idea es que cada uno “piense” distinto al perseguir al jugador:

Dragón A — Perseguidor directo

- Su objetivo es ir hacia la **posición actual del personaje**.
- En cada turno, calcula su movimiento intentando **reducir la distancia** hacia el jugador (por ejemplo, usando una ruta válida por el grid).

Dragón B — Interceptor

- No persigue la posición actual, sino un **punto objetivo adelantado** según la dirección de movimiento del jugador.
- Ejemplo: si el jugador va hacia la derecha, el dragón apunta a una celda “n” pasos por delante (si es válida o, si no lo es, a la mejor alternativa que usted defina).

Dragón C — Estrategia mixta

- Combina dos criterios:
 - A veces persigue directo al jugador.
 - A veces busca **interceptar** o **acorrallar**, usando como referencia la posición del jugador y/o la posición de otro dragón.
- Debe documentar con claridad su regla de decisión (por ejemplo: “si está a más de X celdas, intercepta; si no, persigue directo” o cualquier criterio equivalente).

2.4 Niveles en archivos

Cada nivel del juego debe cargarse desde un archivo `.json` (mínimo 1 nivel, recomendado 3 o más).

El JSON debe representar como mínimo:

- Dimensiones del grid (filas/columnas).
- Distribución de celdas (paredes/piso).
- Posición inicial del jugador.
- Posiciones iniciales de los 3 dragones.
- Ubicación de las 4 llaves.
- Ubicación de la puerta/salida.

Usted define el formato exacto, pero debe ser claro, consistente y documentado.

2.5 Guardar y cargar partida

El juego debe permitir:

- **Guardar** el estado actual en archivo (por ejemplo, JSON u otro formato elegido, pero documentado).
- **Cargar** una partida guardada y continuar exactamente desde ese estado.

El estado debe incluir como mínimo:

- Nivel actual.
- Posición del jugador.
- Posición de los dragones.
- Llaves recolectadas / llaves restantes en el mapa.

- Cualquier información necesaria para continuar sin inconsistencias.

2.6 Replay al terminar el juego

Al completar el juego (escapar con las 4 llaves), el sistema debe mostrar una opción:

- **“Ver repetición”** (replay) de la partida recién terminada.

Requisitos del replay:

- Debe reconstruir, paso a paso, la secuencia de movimientos del jugador y de los dragones.
- Puede reproducirse automáticamente (con velocidad configurable) o paso a paso (tecla para avanzar), pero debe ser claro para el usuario.

2.7) Requisitos técnicos y de estructuras de datos

- El diseño debe aplicar estructuras de datos apropiadas.
- Debe documentar en el README:
 - Qué estructuras de datos usó y por qué.
 - Complejidades esperadas en operaciones clave (por ejemplo: búsqueda de ruta, guardar/cargar, etc.), cuando sea razonable.

2.8) Interfaz gráfica

- Para la interfaz gráfica puede utilizar la librería **Pygame** o **Arcade**.

3) Entregables

1. Código fuente completo (repositorio en Github)
2. Carpeta con los niveles `.json`.
3. Archivo README con:
 - Instrucciones de ejecución.
 - Formato del JSON de niveles.
 - Explicación del comportamiento de cada dragón.
 - Estructuras de datos utilizadas.
4. Bitácora de prompts utilizados con el/los LLM de su elección (ChatGPT, Copilot, Cloude, Gemini, etc.)